

گزارش سؤال چهارم

پیاده‌سازی AES در حالت‌های ECB و CBC

محمد امانلو - ۸۴۰۱۰۰۰۸۱۰۱

۱۵ اردیبهشت ۱۴۰۵

۱ توضیح کلی الگوریتم AES

الگوریتم AES یک الگوریتم رمزنگاری متقارن است. یعنی برای رمزنگاری و رمزگشایی از یک کلید مشترک استفاده می‌شود. در این تمرین از کلید ۱۲۸ بیتی استفاده کردم. چون هر بایت ۸ بیت است، کلید ۱۲۸ بیتی یعنی کلیدی که طول آن ۱۶ بایت باشد. الگوریتم AES داده را به صورت بلوک‌های ۱۲۸ بیتی پردازش می‌کند. یعنی اندازه‌ی هر بلوک در AES برابر ۱۶ بایت است. پس اگر طول متن اولیه مضرب از ۱۶ بایت نباشد، باید قبل از رمزنگاری به آن padding اضافه شود تا طول آن برای رمزنگاری بلوکی مناسب شود. در این تمرین من دو حالت زیر را پیاده‌سازی کردم:

۱. حالت ECB

۲. حالت CBC

۲ کتابخانه‌های استفاده‌شده

در ابتدای برنامه، کتابخانه‌های لازم را وارد کردم:

```
1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 from Crypto.Random import get_random_bytes
```

کتابخانه‌ی AES برای ساختن شیء رمزنگاری استفاده شده است. تابع‌های pad و unpad هم برای اضافه کردن و حذف کردن padding استفاده شده‌اند. همچنین از تابع get_random_bytes برای ساختن IV تصادفی در حالت CBC استفاده کردم.

۳ تعریف کلید، IV و متن اولیه

در این بخش، مقادیر اصلی مورد نیاز برای اجرای برنامه را مشخص کردم. ابتدا کلید رمزنگاری را به صورت زیر تعریف کردم:

```
1 key = b"NetworkSecKey128"
```

این کلید ۱۶ بایت است. از آنجایی که هر بایت ۸ بیت دارد، طول کلید برابر است با:

$$16 \times 8 = 128\text{bits}$$

پس این کلید برای اجرای AES-128 مناسب است.
در ادامه برای حالت CBC، تابع `get_random_bytes` را از کتابخانه `Crypto.Random` وارد کردم:

```
1 from Crypto.Random import get_random_bytes
```

سپس بردار آغازین یا همان IV را به صورت تصادفی و با طول ۱۶ بایت ساختم:

```
1 iv = get_random_bytes(16)
```

چون اندازه‌ی هر بلوک در الگوریتم AES برابر ۱۶ بایت است، طول IV در حالت CBC هم باید ۱۶ بایت باشد. تصادفی بودن IV باعث می‌شود که اگر برنامه چند بار اجرا شود، خروجی رمزنگاری در حالت CBC می‌تواند در هر اجرا متفاوت باشد. این رفتار طبیعی است، چون در CBC، بلوک اول به مقدار IV وابسته است. متن اولیه یا Plaintext را هم به صورت زیر تعریف کردم:

```
1 plaintext = b"This is a sample plaintext for AES ECB and CBC modes."
```

این همان متنی است که در برنامه ابتدا رمزنگاری می‌شود و سپس خروجی رمز شده دوباره رمزگشایی می‌شود تا بررسی شود که متن نهایی با همین متن اولیه برابر است یا نه.
در نهایت اندازه‌ی بلوک الگوریتم AES را هم از خود کتابخانه گرفتم:

```
1 block_size = AES.block_size
```

مقدار `AES.block_size` برابر ۱۶ بایت است و از آن برای انجام padding و unpadding استفاده کردم.

۴ توضیح Padding

چون AES فقط روی بلوک‌های ۱۶ بیتی کار می‌کند، اگر طول متن اولیه مضربی از ۱۶ نباشد، باید قبل از رمزنگاری به آن padding اضافه شود.
در برنامه من این کار را با تابع `pad` انجام دادم:

```
1 pad(plaintext, AES.block_size)
```

تابع `AES.block_size` مقدار ۱۶ را برمی‌گرداند. پس متن اولیه قبل از رمزنگاری به اندازه‌ی padding می‌گیرد که طول آن مضرب ۱۶ شود.
بعد از رمزگشایی هم باید padding حذف شود. برای این کار از تابع `unpad` استفاده کردم:

```
1 unpad(decrypted_data, AES.block_size)
```

اگر `unpad` انجام نشود، متن رمزگشایی‌شده چند بایت اضافه در انتهای خودش دارد و دقیقاً با متن اولیه برابر نمی‌شود.

۵ پیاده‌سازی حالت ECB

۱.۵ توضیح حالت ECB

در حالت ECB هر بلوک متن آشکار به صورت مستقل رمز می‌شود. یعنی رمز شدن هر بلوک به بلوک قبلی یا بعدی وابسته نیست.

فرمول رمزنگاری در این حالت به شکل زیر است:

$$C_i = E_K(P_i)$$

و فرمول رمزگشایی هم به شکل زیر است:

$$P_i = D_K(C_i)$$

در این رابطه‌ها، P_i بلوک متن آشکار، C_i بلوک متن رمز شده، K کلید، E_K تابع رمزنگاری و D_K تابع رمزگشایی است.

مزیت ECB این است که ساده است و هر بلوک را جداگانه رمز می‌کند. اما ضعف مهم آن این است که اگر دو بلوک متن آشکار یکسان باشند، دو بلوک متن رمز شده هم یکسان می‌شوند. پس این حالت می‌تواند الگوی داده را نشان بدهد و برای داده‌های واقعی و طولانی مناسب نیست.

۲.۵ رمزنگاری در ECB

برای رمزنگاری در حالت ECB ابتدا یک شیء رمزنگاری ساختیم:

```
1 ecb_encrypt_cipher = AES.new(key, AES.MODE_ECB)
```

بعد متن اولیه را padding کردم و رمزنگاری را انجام دادم:

```
1 ciphertext_ecb = ecb_encrypt_cipher.encrypt(pad(plaintext, block_size))
```

خروجی این قسمت به صورت بایت است. برای اینکه خروجی قابل خواندن و قابل قرار دادن در گزارش باشد، آن را به قالب هگزادسیمال تبدیل کردم.

۳.۵ رمزگشایی در ECB

برای رمزگشایی، دوباره یک شیء AES در حالت ECB ساختیم:

```
1 ecb_decrypt_cipher = AES.new(key, AES.MODE_ECB)
```

بعد Ciphertext را رمزگشایی کردم و padding را حذف کردم:

```
1 decrypted_plaintext_ecb = unpad(  
2     ecb_decrypt_cipher.decrypt(ciphertext_ecb),  
3     block_size  
4 )
```

در آخر هم بررسی کردم که متن رمزگشایی شده با متن اولیه برابر باشد.

۶ پیاده‌سازی حالت CBC

۱.۶ توضیح حالت CBC

در حالت CBC هر بلوک متن آشکار قبل از رمز شدن با Ciphertext بلوک قبلی XOR می‌شود. برای بلوک اول چون Ciphertext قبلی وجود ندارد، از IV استفاده می‌شود. فرمول رمزنگاری بلوک اول:

$$C_1 = E_K(P_1 \oplus IV)$$

فرمول رمزنگاری بلوک‌های بعدی:

$$C_i = E_K(P_i \oplus C_{i-1})$$

فرمول رمزگشایی بلوک اول:

$$P_1 = D_K(C_1) \oplus IV$$

فرمول رمزگشایی بلوک‌های بعدی:

$$P_i = D_K(C_i) \oplus C_{i-1}$$

در حالت CBC، برخلاف ECB، بلوک‌ها به هم وابسته هستند. همین باعث می‌شود اگر دو بلوک متن آشکار یکسان باشند، معمولاً خروجی رمز شده‌ی آن‌ها یکسان نشود.

۲.۶ رمزنگاری در CBC

برای رمزنگاری در حالت CBC، یک شیء AES با کلید و IV ساختم:

```
1 cbc_encrypt_cipher = AES.new(key, AES.MODE_CBC, iv)
```

بعد متن اولیه را padding کردم و رمزنگاری را انجام دادم:

```
1 ciphertext_cbc = cbc_encrypt_cipher.encrypt(pad(plaintext, block_size))
```

در این حالت، خروجی رمزنگاری علاوه بر متن و کلید، به IV هم وابسته است.

۳.۶ رمزگشایی در CBC

برای رمزگشایی در حالت CBC باید از همان IV استفاده شود. اگر IV اشتباه باشد، رمزگشایی درست انجام نمی‌شود. کد رمزگشایی به صورت زیر است:

```
1 cbc_decrypt_cipher = AES.new(key, AES.MODE_CBC, iv)
2
3 decrypted_plaintext_cbc = unpad(
4     cbc_decrypt_cipher.decrypt(ciphertext_cbc),
5     block_size
6 )
```

بعد از رمزگشایی، padding را حذف کردم و متن نهایی را با متن اولیه مقایسه کردم.

۷ خروجی اجرای برنامه

۸ تحلیل خروجی‌ها

۱.۸ بررسی Plaintext اولیه

در ابتدای خروجی، متن اولیه نمایش داده شده است:

```

PS C:\Users\A\Desktop\New folder (2)> & C:\Users\A\AppData\Local\Programs\Python\Python310\python.exe "c:/Users/a/Desktop/New folder (2)/Q4.py"
Original Plaintext:
This is a sample plaintext for AES ECB and CBC modes.

Ciphertext in ECB mode:
3478CCE7F937D60AFDCE593BA481D910E7C94404F4E7581BECB5C38F4C49BB34BDFBFFB10BD142
2F165A3A87E1B30922F12BD57C277CE2DA05C356DFFF43E0B4

Decrypted Plaintext in ECB mode:
This is a sample plaintext for AES ECB and CBC modes.

Initialization Vector (IV) in CBC mode:
496E697469616C697A6174696F6E5665

Ciphertext in CBC mode:
A5373101279D576BC50FD692B415B7AE1D286B983068851535F33DF1CA680D148F612A6711C9D7
EB8D147C4A0C629176E95793F608C708862BC1BE22A0827959

Decrypted Plaintext in CBC mode:
This is a sample plaintext for AES ECB and CBC modes.

ECB decryption is correct:
True

CBC decryption is correct:
True

```

شکل ۱: اسکرین شات اول خروجی اجرای برنامه در ترمینال

This is a sample plaintext for AES ECB and CBC modes.

این دقیقاً همان متنی است که در برنامه به عنوان Plaintext تعریف کرده بودم. پس برنامه متن ورودی را درست گرفته و درست نمایش داده است.

۲۰۸ تحلیل خروجی ECB

خروجی رمزنگاری در حالت ECB به صورت زیر است:

3478CCE7F937D60AFDCE593BA481D910E7C94404F4E7581BECB5C38F4C49BB34BDFBFFB10BD1422F165A3A87E1B30922F12BD57C277CE2DA05C356DFFF43E0B4

این مقدار به صورت هگزادسیمال نمایش داده شده است. چون AES خروجی را به شکل بایت تولید می کند، نمایش هگزادسیمال باعث می شود که Ciphertext خوانا تر شود. بعد از رمزگشایی در حالت ECB، خروجی زیر به دست آمد:

This is a sample plaintext for AES ECB and CBC modes.

این متن دقیقاً با متن اولیه برابر است. پس پیاده سازی حالت ECB درست انجام شده است. همچنین خط زیر در خروجی این موضوع را نشان می دهد:

ECB decryption is correct:
True

یعنی عبارت زیر در برنامه مقدار True داده است:

```
1 decrypted_plaintext_ecb == plaintext
```

پس رمزنگاری و رمزگشایی در حالت ECB درست کار کرده است.

```
Original Plaintext:
This is a sample plaintext for AES ECB and CBC modes.

Ciphertext in ECB mode:
3478CCE7F937D60AFDCE593BA481D910E7C94404F4E7581BECB5C38F4C49B834BDFBFFB10BD1422F165A3A87E1B30922F12BD57C277CE2DA05C356DFFF43E0B4

Decrypted Plaintext in ECB mode:
This is a sample plaintext for AES ECB and CBC modes.

Initialization Vector (IV) in CBC mode:
DEA3A16431AAD6D3F0DDF610E24E48A8

Ciphertext in CBC mode:
F268F01EA208C0F1A08CF784B655997ADED338DAF224BA6A25C70C884AAF1004C6F1087DAFB0844114B387F254F6011A5F69A19C5DB1E0EAEDCD16F8588DC103

Decrypted Plaintext in CBC mode:
This is a sample plaintext for AES ECB and CBC modes.

ECB decryption is correct:
True

CBC decryption is correct:
True
PS C:\Users\A\Desktop\New folder (2)>
```

شکل ۲: اسکرین‌شات دوم خروجی اجرای برنامه در ترمینال

۳۰۸ تحلیل IV در CBC

در خروجی، مقدار IV برابر است با:

```
DEA3A16431AAD6D3F0DDF610E24E48A8
```

این مقدار ۳۲ رقم هگزادسیمال دارد. چون هر دو رقم هگزادسیمال برابر یک بایت است، پس طول IV برابر است با:

$$\frac{32}{2} = 16 \text{ bytes}$$

پس IV طول درستی دارد، چون اندازه‌ی بلوک AES هم ۱۶ بایت است.

۴۰۸ تحلیل خروجی CBC

خروجی رمزنگاری در حالت CBC به شکل زیر است:

```
F268F01EA208C0F1A08CF784B655997ADED338DAF224BA6A25C70C884AAF1004C6F1087DAFB0844114B387F254F6011A5F69A19C5DB1E0EAEDCD16F8588DC103
```

این خروجی با خروجی ECB فرق دارد. این موضوع کاملاً طبیعی است، چون در حالت CBC قبل از رمز شدن هر بلوک، آن بلوک با مقدار قبلی ترکیب می‌شود. برای بلوک اول هم از IV استفاده می‌شود. پس حتی اگر متن اولیه و کلید یکسان باشند، خروجی CBC با خروجی ECB فرق می‌کند. بعد از رمزگشایی در حالت CBC، خروجی زیر به دست آمد:

```
This is a sample plaintext for AES ECB and CBC modes.
```

این خروجی دقیقاً با متن اولیه برابر است. پس رمزگشایی CBC هم درست انجام شده است. همچنین برنامه این نتیجه را چاپ کرده است:

```
CBC decryption is correct:
True
```

یعنی عبارت زیر مقدار True داشته است:

```
1 decrypted_plaintext_cbc == plaintext
```

بنابراین پیاده‌سازی حالت CBC هم درست است.

۹ بررسی دستی درستی نتایج

برای اینکه فقط به خروجی True اکتفا نکنم، چند مورد را دستی هم بررسی کردم.

۱۰.۹ بررسی طول کلید

کلید برنامه این بود:

```
b"NetworkSecKey128"
```

این رشته ۱۶ کاراکتر دارد. چون هر کاراکتر در این حالت یک بایت است، طول کلید برابر ۱۶ بایت می‌شود:

$$16 \times 8 = 128\text{bits}$$

پس کلید واقعاً ۱۲۸ بیتی است و برای AES-128 درست است.

۲۰.۹ بررسی طول IV

مقدار IV در خروجی این بود:

```
DEA3A16431AAD6D3F0DDF610E24E48A8
```

این رشته ۳۲ رقم هگزادسیمال دارد. چون هر ۲ رقم هگزادسیمال برابر ۱ بایت است، پس داریم:

$$\frac{32}{2} = 16\text{bytes}$$

پس طول IV برابر ۱۶ بایت است و برای حالت CBC درست است.

۳۰.۹ بررسی طول Ciphertext

خروجی Ciphertext در حالت ECB برابر بود با:

```
3478CCE7F937D60AFDCE593BA481D910E7C94404F4E7581BECB5C38F4C49BB34BDFBFB10BD1422F165A3A87E1B30922F12BD57C277CE2DA05C356DFFF43E0B4
```

این خروجی ۱۲۸ رقم هگزادسیمال دارد. بنابراین طول آن بر حسب بایت برابر است با:

$$\frac{128}{2} = 64\text{bytes}$$

طول متن اولیه ۵۳ بایت است. چون AES باید روی بلوک‌های ۱۶ بیتی کار کند، طول متن بعد از padding باید به نزدیک‌ترین مضرب ۱۶ برسد. نزدیک‌ترین مضرب ۱۶ بعد از ۵۳ برابر ۶۴ است:

$$64 - 53 = 11$$

پس برنامه ۱۱ بایت padding اضافه کرده است. در نتیجه اینکه طول Ciphertext برابر ۶۴ بایت شده، کاملاً درست و منطقی است.

همین موضوع برای خروجی CBC هم برقرار است. خروجی CBC هم ۱۲۸ رقم هگزادسیمال دارد، یعنی ۶۴ بایت است.

۴.۹ بررسی برابر بودن متن اولیه و متن رمزگشایی شده

متن اولیه:

This is a sample plaintext for AES ECB and CBC modes.

متن رمزگشایی شده در حالت ECB:

This is a sample plaintext for AES ECB and CBC modes.

متن رمزگشایی شده در حالت CBC:

This is a sample plaintext for AES ECB and CBC modes.

هر سه متن دقیقاً یکسان هستند. پس هم رمزنگاری درست انجام شده، هم رمزگشایی، و هم حذف padding درست بوده است.

۵.۹ بررسی تفاوت خروجی ECB و CBC

خروجی ECB:

3478CCE7F937D60AFDCE593BA481D910E7C94404F4E7581BECB5C38F4C49BB34BDFBFFB10BD1422F165A3A87E1B30922F12BD57C277CE2DA05C356DFFF43E0B4

خروجی CBC:

F268F01EA208C0F1A08CF784B655997AED338DAF224BA6A25C70C884AAF1004C6F1087DAFB0844114B387F254F6011A5F69A19C5DB1E0EAEDCD16F8588DC103

این دو مقدار با هم فرق دارند. این تفاوت درست است، چون ECB هر بلوک را مستقل رمز می کند، ولی CBC از IV و زنجیره سازی بین بلوک ها استفاده می کند. بنابراین خروجی CBC به IV هم وابسته است.

۱۰ پاسخ به پرسش های تکمیلی

۱۰.۱ الف) اگر در حالت CBC بردار آغازین IV تغییر کند، چه تغییری در نتیجه خواهد داشت؟

در حالت CBC، بردار آغازین یا همان IV در رمزنگاری بلوک اول استفاده می شود. فرمول رمزنگاری بلوک اول در این حالت به صورت زیر است:

$$C_1 = E_K(P_1 \oplus IV)$$

بنابراین اگر IV تغییر کند، حتی اگر Plaintext و کلید همان قبلی باشند، مقدار ورودی تابع رمزنگاری برای بلوک اول تغییر می کند. در نتیجه Ciphertext بلوک اول یعنی C_1 تغییر می کند. از طرف دیگر در حالت CBC، رمزنگاری هر بلوک بعدی به Ciphertext بلوک قبلی وابسته است:

$$C_i = E_K(P_i \oplus C_{i-1})$$

پس وقتی C_1 تغییر کند، روی C_2 هم اثر می گذارد، و همین وابستگی به بلوک های بعدی هم منتقل می شود. بنابراین اگر در زمان رمزنگاری، IV را تغییر بدهیم، معمولاً کل Ciphertext خروجی در حالت CBC تغییر می کند. در برنامه ی من هم چون در نسخه ی نهایی از IV تصادفی استفاده شده بود، هر بار اجرای برنامه می تواند مقدار IV متفاوتی بدهد. به همین دلیل خروجی Ciphertext در حالت CBC هم در اجراهای مختلف متفاوت می شود. این موضوع نشانه ی خطا نیست، بلکه رفتار طبیعی حالت CBC است. نکته ی مهم دیگر این است که اگر Ciphertext درست باشد ولی در زمان رمزگشایی از IV اشتباه استفاده کنیم، فقط بلوک اول Plaintext خراب می شود. چون در رمزگشایی داریم:

$$P_1 = D_K(C_1) \oplus IV$$

پس اگر IV اشتباه باشد، نتیجه‌ی P_1 اشتباه می‌شود. اما برای بلوک‌های بعدی از Ciphertext بلوک قبلی استفاده می‌شود:

$$P_i = D_K(C_i) \oplus C_{i-1}$$

بنابراین در صورت اشتباه بودن فقط IV هنگام رمزگشایی، بلوک‌های بعدی معمولاً درست باقی می‌مانند. پس نتیجه این است که:

تغییر IV در رمزنگاری CBC باعث تغییر Ciphertext، مخصوصاً از بلوک اول به بعد می‌شود.

و همچنین:

استفاده از IV اشتباه در رمزگشایی CBC معمولاً فقط بلوک اول Plaintext را خراب می‌کند.

۲۰۱۰ ب) اگر در حالت ECB دو بلوک ورودی یکسان وجود داشته باشد، آیا Ciphertext آن‌ها نیز یکسان خواهد بود؟

بله. در حالت ECB اگر دو بلوک ورودی یکسان باشند و با یک کلید یکسان رمز شوند، خروجی Ciphertext آن‌ها هم یکسان خواهد بود.

دلیلش این است که در ECB هر بلوک کاملاً مستقل از بلوک‌های دیگر رمز می‌شود. فرمول رمزنگاری در این حالت به صورت زیر است:

$$C_i = E_K(P_i)$$

یعنی هر بلوک Plaintext فقط با همان کلید وارد تابع رمزنگاری می‌شود. پس اگر داشته باشیم:

$$P_i = P_j$$

و کلید هم یکی باشد، چون تابع رمزنگاری قطعی است، خروجی‌ها هم برابر می‌شوند:

$$E_K(P_i) = E_K(P_j)$$

در نتیجه:

$$C_i = C_j$$

این موضوع یکی از ضعف‌های اصلی حالت ECB است. چون اگر در متن اصلی الگوی تکراری وجود داشته باشد، این الگو در Ciphertext هم تا حدی قابل مشاهده می‌شود. به همین دلیل حالت ECB برای رمزنگاری پیام‌های طولانی یا داده‌هایی که ساختار تکراری دارند مناسب نیست. پس پاسخ نهایی این بخش:

بله، در ECB دو بلوک Plaintext یکسان، Ciphertext یکسان تولید می‌کنند.

دلیل آن هم مستقل بودن رمزنگاری هر بلوک و قطعی بودن تابع رمزنگاری با کلید ثابت است.

۱۱ جمع‌بندی

در این تمرین من الگوریتم AES را با کلید ۱۲۸ بیتی در دو حالت ECB و CBC پیاده‌سازی کردم. برای اینکه متن اولیه قابل رمز شدن با AES باشد، از padding استفاده کردم. در حالت ECB هر بلوک به صورت مستقل رمز شد و در حالت CBC از IV و زنجیره‌سازی بین بلوک‌ها استفاده شد. خروجی برنامه نشان داد که در هر دو حالت، متن رمزگشایی شده دقیقاً با متن اولیه برابر است. نتیجه‌ی نهایی برنامه این بود:

```
ECB decryption is correct:
True

CBC decryption is correct:
True
```

پس پیاده‌سازی هر دو حالت درست انجام شده است. همچنین با بررسی دستی طول کلید، طول IV، طول Ciphertext، مقدار padding و برابر بودن متن اولیه و متن رمزگشایی شده، مشخص شد که خروجی‌های برنامه درست و منطقی هستند.